第4回：スペクトログラムの理解と可視化演習

音声処理と認識

担当：小野原彩香

形式：オンライン同期（Zoom）

📌 Zoom録画を後日公開します。課題はメール/Google Driveでも提出可。

本日の到達目標

1. フーリエ変換の直感的な意味を説明できる
2. DFT・FFTの計算方法を理解し、Pythonで実行できる
3. STFTとスペクトログラムの仕組みを理解する
4. 窓関数の役割を説明できる
5. Pythonでスペクトログラムを描画・読み取れる
6. 有声音・無声音・無音の違いをスペクトログラムで識別できる

音はサイン波の重ね合わせ

自然界の音はどんなに複雑でも、**単純なサイン波（正弦波）の足し算**で表せる。

例：ピアノの「ラ（440 Hz）」は

- 基音：440 Hz のサイン波（最も大きい）
- 倍音：880 Hz・1320 Hz・1760 Hz…（高次の倍音が続く）

$$x(t) = A_1 \sin(2\pi f_1 t) + A_2 \sin(2\pi f_2 t) + A_3 \sin(2\pi f_3 t) + \cdots$$

> **アナロジー：光とプリズム**
> 白色光をプリズムに通すと虹色に分かれる。
> 音をフーリエ変換すると、構成する周波数成分に分かれる。

DFT（離散フーリエ変換）の定義

コンピュータは連続信号を扱えないため、離散信号（デジタル音声）を処理する。

DFTの定義式

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1$$

- $x[n]$：時間領域の離散信号（N 個のサンプル）
- $X[k]$：周波数領域の複素数（k 番目の周波数成分の大きさと位相）
- $|X[k]|$：振幅スペクトル（その周波数の強さ）

対応する周波数：$f_k = \dfrac{k \cdot F_s}{N}$（$F_s$：サンプリング周波数）

FFT（高速フーリエ変換）：DFTの計算量 $O(N^2)$ を $O(N \log N)$ に短縮したアルゴリズム

FFTをPythonで計算してみよう

```
import numpy as np
import matplotlib.pyplot as plt

# サンプリング設定
fs = 44100          # サンプリング周波数（Hz）
duration = 0.1      # 信号の長さ（秒）
t = np.linspace(0, duration, int(fs * duration), endpoint=False)

# 440 Hz と 880 Hz のサイン波を合成
x = np.sin(2 * np.pi * 440 * t) + 0.5 * np.sin(2 * np.pi * 880 * t)

# FFTで周波数解析
X = np.fft.fft(x)
N = len(x)
freqs = np.fft.fftfreq(N, d=1/fs)

# 正の周波数だけ取り出してプロット
positive = freqs > 0
plt.figure(figsize=(10, 4))
plt.plot(freqs[positive], np.abs(X[positive]))
plt.xlabel("周波数（Hz）"); plt.ylabel("振幅")
plt.title("FFTによる周波数スペクトル（440 Hz + 880 Hz）")
plt.xlim(0, 2000); plt.grid(True); plt.tight_layout(); plt.show()
```

440 Hz と 880 Hz に鋭いピークが現れることを確認する

短時間フーリエ変換（STFT）— なぜ「短時間」か

問題点：FFTだけでは不十分

FFTは「信号全体にどんな周波数が含まれているか」はわかるが、**「いつ」その周波数が現れたか**はわからない。

STFTのアイデア：短い区間（フレーム）ごとにFFTをかける

$$\mathrm{STFT}\{x\}(m,k) = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-j\frac{2\pi}{N}kn}$$

- $w[n]$：窓関数（後述）　m：フレーム番号　H：ホップサイズ　N：フレーム長

パラメータ	典型的な値	意味
フレーム長	25 ms（≈ 1024 サンプル @44.1kHz）	各分析窓の長さ
ホップサイズ	10 ms（≈ 512 サンプル @44.1kHz）	フレームのずらし幅
オーバーラップ	50〜75%	フレーム間の重複率

窓関数の役割

スペクトル漏れ（Spectral Leakage）の問題
フレームをそのまま（矩形窓で）切り出すと、端で信号が急に途切れ、本来存在しない周波数成分が発生する。

解決策：端をなだらかに 0 にする窓関数を使う

ハニング窓（Hann窓）の式：

$$w[n] = 0.5\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right)$$

窓関数	特徴	用途
矩形窓	漏れ大・分解能高	周波数が整数倍のときのみ
ハニング窓	バランスが良い	一般的な音声処理（定番）
ハミング窓	ハニングに近い	音声処理（古典的）
ブラックマン窓	漏れ小・分解能低	精密スペクトル測定

スペクトログラムの読み方

STFTの結果を**2次元カラーマップ**として可視化したもの

```
縦軸（Y軸）：周波数（Hz）
  ↑ 高い音
  |
  ↓ 低い音
  ──────────────────────→ 横軸（X軸）：時間（秒）

色：そのフレーム・その周波数のエネルギー強度
  明るい色（黄・白）→ 強い
  暗い色（紺・黒）  → 弱い
```

対数スケール（dB）を使う理由

$$P_{\mathrm{dB}} = 10 \log_{10} \frac{P}{P_{\mathrm{ref}}}$$

- 人間の聴覚に近いスケール → 音声処理では常用
- 線形スケールだと低音域が密集して見えにくい

有声音・無声音・無音のスペクトログラムの違い

種類	例	スペクトログラムの特徴
有声音	あ・い・う・え・お・な・ま	規則的な水平縞模様（調波構造）
無声音	さ・し・す・せ・そ・た・か	高周波に広帯域のノイズ状成分
無音	発話の間	全体的に暗い（エネルギーほぼゼロ）

有声音の調波構造（Harmonic Structure）

- 基本周波数 F_0（男性：100〜150 Hz、女性：200〜300 Hz）
- F_0 の整数倍（$2F_0, 3F_0, 4F_0, \ldots$）に強いエネルギー → 等間隔の水平縞

無声音の特徴

- 声帯振動なし → 調波構造がない
- 例：「さ（/s/）」は 3000〜8000 Hz に広帯域ノイズ

日本語母音「あいうえお」のフォルマント

声道（口や喉の形）により強調される周波数帯域を**フォルマント**という。

母音	F1 (Hz)	F2 (Hz)	口の形
あ /a/	800	1200	大きく開く
い /i/	300	2500	横に広げる
う /u/	400	1400	前に突き出す（やや）
え /e/	500	1900	中間
お /o/	500	800	丸く突き出す

スペクトログラムで見分けるポイント

- 「あ」：F1 が高い（800 Hz）→ 低〜中域に強い縞
- 「い」：F2 が非常に高い（2500 Hz）→ 高域に明るい縞
- 「お」：F2 が低い（800 Hz）→ 低域に集中

演習1：スペクトログラムの描画（Colab）

```
import librosa, librosa.display
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio, display

# ライブラリ付属のサンプル音声を使用
audio_path = librosa.example('trumpet')
y, sr = librosa.load(audio_path, sr=None)
print(f"サンプリングレート: {sr} Hz  /  長さ: {len(y)/sr:.2f} 秒")
display(Audio(y, rate=sr))

# STFTを計算してスペクトログラムを描画
n_fft, hop_length = 2048, 512
D = librosa.stft(y, n_fft=n_fft, hop_length=hop_length, window='hann')
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

fig, ax = plt.subplots(figsize=(12, 4))
img = librosa.display.specshow(S_db, sr=sr, hop_length=hop_length,
                               x_axis='time', y_axis='log',
                               ax=ax, cmap='magma')
ax.set_title("スペクトログラム（対数周波数スケール、dB）")
fig.colorbar(img, ax=ax, format='%+.0f dB')
plt.tight_layout(); plt.show()
```

演習2：母音「あいうえお」の比較（Colab）

```
import numpy as np, librosa, librosa.display, matplotlib.pyplot as plt

def make_vowel(f0, formants, duration=0.5, sr=22050):
    t = np.linspace(0, duration, int(sr * duration), endpoint=False)
    signal = sum((1/k) * np.sin(2*np.pi*f0*k*t) for k in range(1, 10))
    for f_f in formants:
        signal += 0.5 * np.sin(2*np.pi*f_f*t)
    return signal / np.max(np.abs(signal))

vowels = {
    'あ': [800, 1200, 2600], 'い': [300, 2500, 3200],
    'う': [400, 1400, 2400], 'え': [500, 1900, 2700], 'お': [500, 800, 2400]
}
f0, sr, n_fft, hop = 150, 22050, 1024, 256
fig, axes = plt.subplots(1, 5, figsize=(18, 4))

for col, (name, fmts) in enumerate(vowels.items()):
    sig = make_vowel(f0, fmts, sr=sr)
    D = librosa.stft(sig, n_fft=n_fft, hop_length=hop)
    S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)
    librosa.display.specshow(S_db, sr=sr, hop_length=hop,
                             x_axis='time', y_axis='hz',
                             ax=axes[col], cmap='magma')
    axes[col].set_title(f"「{name}」"); axes[col].set_ylim(0, 5000)

fig.suptitle("日本語母音「あいうえお」のスペクトログラム比較（合成音声）")
plt.tight_layout(); plt.show()
```

演習3：自分の声を録音して可視化（Colab）

```
from google.colab import output
from IPython.display import Javascript

record_js = """
async function recordAudio() {
  const stream = await navigator.mediaDevices.getUserMedia({audio: true});
  const recorder = new MediaRecorder(stream);
  const chunks = [];
  return new Promise((resolve) => {
    recorder.ondataavailable = e => chunks.push(e.data);
    recorder.onstop = () => {
      const blob = new Blob(chunks, {type: 'audio/webm'});
      const reader = new FileReader();
      reader.onloadend = () => resolve(reader.result);
      reader.readAsDataURL(blob);
    };
    recorder.start();
    setTimeout(() => { recorder.stop(); stream.getTracks().forEach(t=>t.stop()); }, 3000);
  });
}
const result = await recordAudio();
google.colab.kernel.invokeFunction('notebook.set_audio', [result], {});
"""
recorded_audio = {}
def set_audio(d): recorded_audio['data'] = d; print("録音完了！")
output.register_callback('notebook.set_audio', set_audio)
display(Javascript(record_js))
print("3秒間「あいうえお」などを話してください…")
```

録音後は波形・スペクトログラム・メルスペクトログラムを比較して観察する

演習課題（授業中）

課題1（個人）：スペクトログラムを読み取ろう

librosa のサンプル音声を使い、以下を確認する：

1. 有声音と無声音の区間を時間で特定する
2. 基本周波数（F_0）をスペクトログラムから目視で推定する
3. 窓関数を矩形窓（`window='boxcar'`）と ハニング窓（`window='hann'`）に変えてスペクトログラムの違いを観察する

課題2（任意）：自分の声で比較

1. 「あ」と「い」を約1秒ずつ発声して録音
2. スペクトログラムを描画し、F1・F2 の位置の違いをスクリーンショットに矢印で示す
3. Zoomチャットに貼り付けて共有、またはメールで送る

次回（第5回）の予告

形式：動画回（非同期）
テーマ：MFCC ― 音声認識の最重要特徴量

> スペクトログラムの情報を人間の聴覚特性に合わせて圧縮する。
> これが音声認識の最重要特徴量 MFCC です！

予習課題（やれる範囲で）

- 「メル尺度」「ケプストラム」という言葉を調べておく
- 今日描いたスペクトログラムを見て「どの部分が人間にとって重要な情報か」を考えておく

第4回のまとめ

概念	ポイント
フーリエ変換	音 = サイン波の重ね合わせ。FFTで効率的に計算（$O(N \log N)$）
DFTの定義	$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn}$
STFT	短い区間ごとにFFT → 時間変化を追える
窓関数	スペクトル漏れを防ぐ。ハニング窓が音声処理の定番
スペクトログラム	横=時間、縦=周波数、色=強度の2次元マップ（dBスケール推奨）
有声音	等間隔の調波構造（縞模様）で F_0 と倍音が見える
無声音	高域に広帯域ノイズ（縞模様なし）

課題：演習のスクリーンショットを翌週末 23:59 までに Google Drive または メールで提出